EL6: Data sources

data.table, SQL, DuckDB, parquett, SAS, JSON, API

Reading

- [1, ch. 2] on Databases: it is important to understand some basic concepts. You might, however, ignore sections about "(Hyper)graph databases". Try to understand the SQL code even though you might not need to write such code on your own.
- H. Wickham, M. Çetinkaya-Rundel, and G. Grolemund [2] chap 21 describes how to connect to a database from R.
- S. R. Fenk, K. Furu, and I. J. Bakken [3] describe why the current common practice with register data delivery in SAS format should be replaced by parquet files. Until this has happened, we often need to use SAS for data extraction.
- [4] introduces the `data.table` syntax, its general form, how to *subset* rows, *select and compute* on columns, and perform aggregations *by group*.

Recommended references:

- V. Arel-Bundock [5] provides a good comparison between base R, dplyr and data.table.

Optional practice

- We won't focus on SQL in this course but it might still be good to know some of the basics. A useful resources for your own practicing: SQLzoo
- The SQL Tutorial for Data Analysis is another option

.sas7bdat

- It is common to receive large data files in SAS format (`.sas7bdat`)
- Many govermental agencies are using SAS internally
- SAS is great for handling big data sets (no need to read everything into memory)
- But I don't know how to use SAS 😩
- Possible to read medium-sized SAS files to R by `haven::read_sas()`
- But may not work for larger files
- Best practice for current SAS version (9.4) is to export only nececary data to csv
  - SAS Viya (next version) will be more flexible
- You need a SAS license (expensive) as well as some SAS syntax for this
  - But you might ask ChatGPT etc to generate such code for you
- The exported csv file can later be read into R

Databases

- a database as a collection of "tables" (tabular data frames)
- Differences between data frames and database tables:
  - Database tables are stored on disk and can be arbitrarily large.
  - Data frames are stored in memory
  - Database tables almost always have indexes; makes it possible to quickly find rows of interest
    - Data frames and tibbles don't have indexes, but data.tables do, which is one of the reasons that they're so fast.
  - No fixed row order in a table

Row vs column oriented

- Most classical databases are optimized for rapidly **collecting** data, **not analyzing existing data**.
- These databases are called row-oriented because the data is stored row-by-row, rather than column-by-column like R.
- More recently, there's been much development of column-oriented databases that make analyzing the existing data much faster.

DBMS

Databases are run by database management systems (DBMS's for short), which come in three basic forms

Client-server

run on a powerful central server, which you connect to from your computer (the client). For example PostgreSQL, MariaDB, SQL Server, and Oracle. Those are sometimes used in our field but if so, you will hopefully get some initial help from the IT department who will provide acess details etc.

Cloud

like Snowflake, Amazon's RedShift, and Google's BigQuery, are similar to client server DBMS's, but they run in the **cloud**. This means that they can easily handle extremely large datasets and can automatically provide more compute resources as needed. Commercial cloud products are less likely to be used for sensitive health data within our field (so we will not focus on these).

In-process

like SQLite or duckdb, run entirely on your computer. They're great for working with large datasets where you're the primary user. We focus on DuckDB in this course!

SQL

- Different database systems may use different query languages.
- The **Structured Query Language (SQL)** is the most widely used for relational databases.

- Defined by international standards (ANSI/ISO).

What is SQL used for?

SQL is used to:

- Retrieve data (`SELECT`)
- Filter data (`WHERE`)
- Aggregate data (`GROUP BY`)
- Combine tables (`JOIN`)
- Modify data (`INSERT`, `UPDATE`, `DELETE`)

SQL Dialects

- SQL is a standard, but implementations differ slightly.
- These variations are called **dialects**.
  - **T-SQL** (Microsoft SQL Server)
  - **PL/SQL** (Oracle)
  - PostgreSQL SQL dialect
  - MySQL SQL dialect

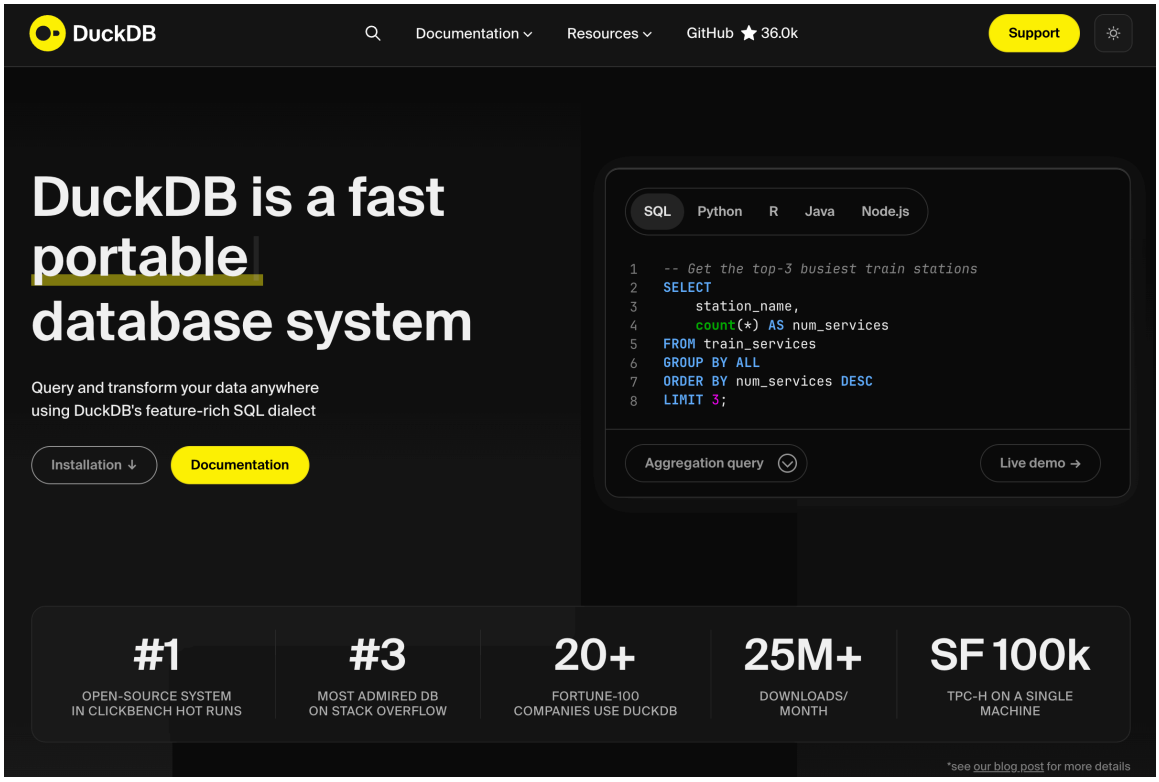
Most core functionality is similar across systems.

A Simple Example

```sql
SELECT name, age
FROM students
WHERE age >= 18
ORDER BY age;
```

This query:

- Selects two columns
- Filters rows
- Sorts the result



DuckDB

- Free and open source (backed by foundation)
- Column based
- Very easy to set up
- Very efficient!
- SQL is an option but not a requirement
  - Integration with tidyverse in R (dbplyr or duckplyr)
- Can be used both with disk data and data in RAM

Example

```
# pak::pak("tidyverse/duckplyr")
library(duckplyr)
```

```
Loading required package: dplyr
```

```

Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag
```

```
The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
The duckplyr package is configured to fall back to dplyr when it encounters an
incompatibility. Fallback events can be collected and uploaded for analysis to
guide future development. By default, data will be collected but no data will
be uploaded.
ℹ Automatic fallback uploading is not controlled and therefore disabled, see
  `?duckplyr::fallback()`.
✔ Number of reports ready for upload: 15.
→ Review with `duckplyr::fallback_review()`, upload with
  `duckplyr::fallback_upload()`.
ℹ Configure automatic uploading with `duckplyr::fallback_config()`.
```

```
✔ Overwriting dplyr methods with duckplyr methods.
ℹ Turn off with `duckplyr::methods_restore()`.
```

```
conflicted::conflicts_prefer(dplyr::filter)
```

```
[conflicted] Will prefer dplyr::filter over any other package.
```

```
# Extra tools to connect to internet data
db_exec("INSTALL httpfs")
db_exec("LOAD httpfs")

# Use some online data which is to big to keep in memory
year <- 2022:2024
base_url <- "https://blobs.duckdb.org/flight-data-partitioned/"
files <- paste0("Year=", year, "/data_0.parquet")
urls <- paste0(base_url, files)
tibble(urls)
```

```
# A tibble: 3 × 1
  urls
  <chr>
1 https://blobs.duckdb.org/flight-data-partitioned/Year=2022/data_0.parquet
```

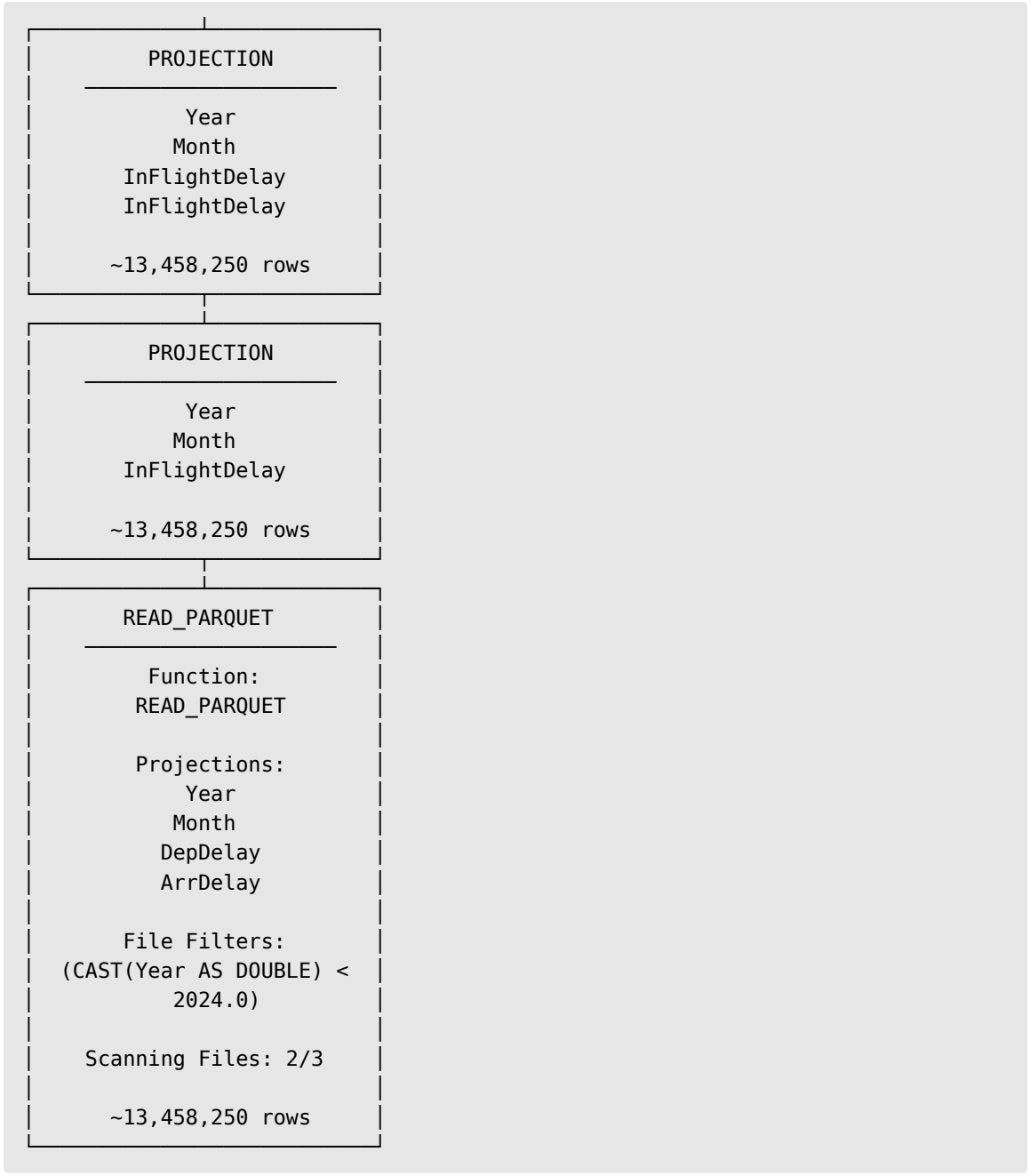
```
2 https://blobs.duckdb.org/flight-data-partitioned/Year=2023/data_0.parquet
3 https://blobs.duckdb.org/flight-data-partitioned/Year=2024/data_0.parquet
```

```
# Connect to the data (without downloading)
flights <- read_parquet_duckdb(urls)
# nrow(flights) # It's not in memory!
count(flights, Year) # processing outside memory
```

```
# A duckplyr data frame: 2 variables
   Year       n
  <dbl>   <int>
1  2022 6729125
2  2023 6847899
3  2024 3461319
```

```
# Complex queries can be executed on the remote data. Note how only the
relevant columns are fetched and the 2024 data isn't even touched, as it's not
needed for the result.
out <-
  flights |>
  mutate(InFlightDelay = ArrDelay - DepDelay) |>
  summarize(
    .by = c(Year, Month),
    MeanInFlightDelay = mean(InFlightDelay, na.rm = TRUE),
    MedianInFlightDelay = median(InFlightDelay, na.rm = TRUE),
  ) |>
  filter(Year < 2024)

out |>
  explain()
```

```
┌─────────────────────────────┐
│       HASH_GROUP_BY         │
│    ─────────────────────    │
│          Groups:            │
│             #0              │
│             #1              │
│                             │
│        Aggregates:          │
│          mean(#2)           │
│         median(#3)          │
│                             │
│      ~13,458,249 rows       │
└──────────────┬──────────────┘
```



DuckDB

- DuckDB is an **embedded analytical SQL database**.
- It supports standard SQL
- No server installation is required.
- Designed for analytical workloads

DuckDB and Data Files

DuckDB can query files directly:

- Parquet
- CSV
- Arrow
- JSON

No data import is required.

Example

- Let's say you used SAS to export your data to a csv file
- DuckDB can query the relevant data from that file directly and collect only the data you actually need!

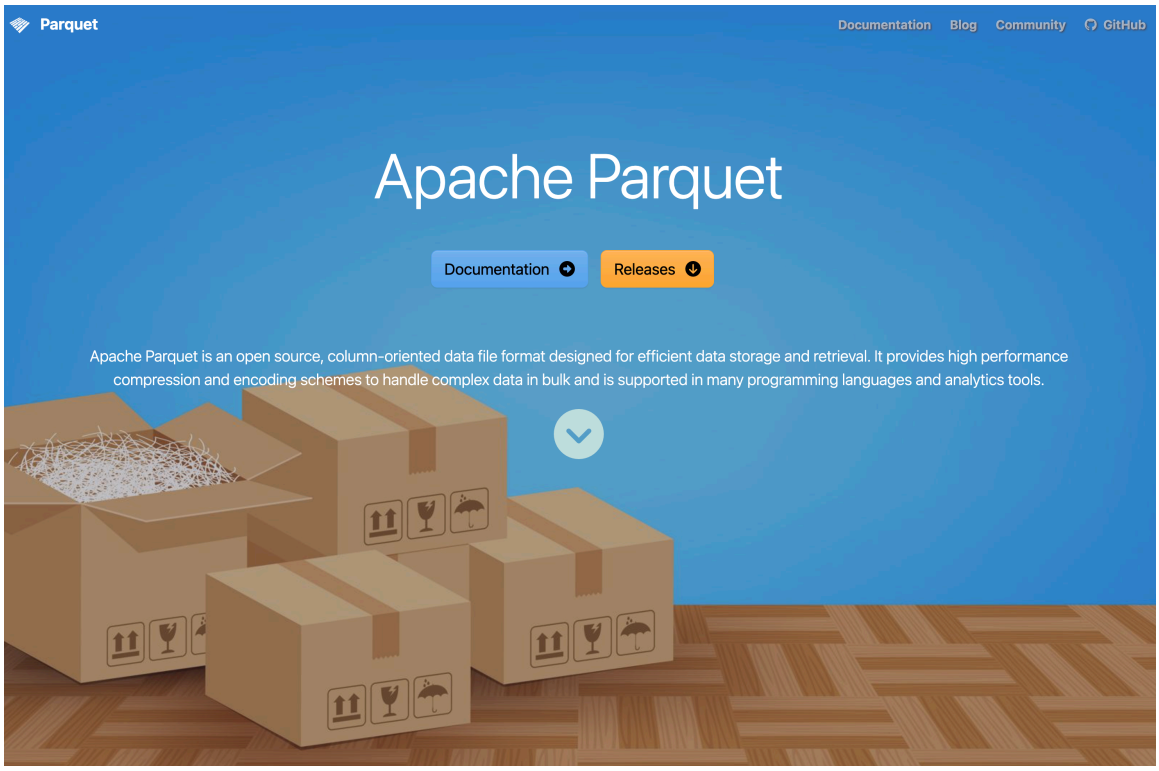
```
csv_file <- tempfile(fileext = ".csv")
write.csv(NHANES::NHANES, csv_file)

read_csv_duckdb(csv_file) |>
  filter(Age >= 18) |>
  summarise(BMI_mean = mean(as.numeric(BMI), na.rm = TRUE), .by = Gender)
```

```
Warning: There were 2 warnings in `summarise()`.
The first warning was:
ℹ In argument: `BMI_mean = mean(as.numeric(BMI), na.rm = TRUE)`.
ℹ In group 1: `Gender = "male"`.
Caused by warning in `mean()`:
! NAs introduced by coercion
ℹ Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```
# A duckplyr data frame: 2 variables
  Gender BMI_mean
  <chr>     <dbl>
1 male       28.7
2 female     28.7
```

Parquet

- From Apache Arrow
- Open format
- flat files with possible hierarchical structure by transparent structure of folders and filenames
- very fast to read



Example

- So we might work with CSV + DuckDB but it is even more efficient if we could convert the CSV to a parquete file
- The solution below reads all CSV data and then export it to a new parquet file
- This means that all data must still fit into memory
  - If it doesn't, do it chunk-wise (piece by piece, 1 M rows at the time or similar)

```
# install.packages("arrow")
library(arrow)
parquet_file <- tempfile(fileext = ".parquet")
read_csv_arrow(csv_file) |>
  write_parquet(parquet_file)

# Now
read_parquet_duckdb(parquet_file)
```

```
# A duckplyr data frame: 77 variables
      C0    ID SurveyYr Gender   Age AgeDecade AgeMonths Race1 Race3
Education
   <int> <int> <chr>    <chr>  <int> <chr>         <int> <chr> <chr>
<chr>
 1     1 51624 2009_10  male      34 " 30-39"        409 White <NA>  High
```

```
School
 2     2 51624 2009_10  male      34 " 30-39"        409 White <NA>  High
School
 3     3 51624 2009_10  male      34 " 30-39"        409 White <NA>  High
School
 4     4 51625 2009_10  male       4 " 0-9"           49 Other <NA>
<NA>
 5     5 51630 2009_10  female    49 " 40-49"        596 White <NA>  Some
Colle…
 6     6 51638 2009_10  male       9 " 0-9"          115 White <NA>
<NA>
 7     7 51646 2009_10  male       8 " 0-9"          101 White <NA>
<NA>
 8     8 51647 2009_10  female    45 " 40-49"        541 White <NA>  College
Gr…
 9     9 51647 2009_10  female    45 " 40-49"        541 White <NA>  College
Gr…
10    10 51647 2009_10  female    45 " 40-49"        541 White <NA>  College
Gr…
# ℹ more rows
# ℹ 67 more variables: MaritalStatus <chr>, HHIncome <chr>, HHIncomeMid <int>,
#   Poverty <dbl>, HomeRooms <int>, HomeOwn <chr>, Work <chr>, Weight <dbl>,
#   Length <dbl>, HeadCirc <dbl>, Height <dbl>, BMI <dbl>,
#   BMICatUnder20yrs <chr>, BMI_WHO <chr>, Pulse <int>, BPSysAve <int>,
#   BPDiaAve <int>, BPSys1 <int>, BPDia1 <int>, BPSys2 <int>, BPDia2 <int>,
#   BPSys3 <int>, BPDia3 <int>, Testosterone <dbl>, DirectChol <dbl>, …
```

tidy data

- `duckplyr` let's you use the familiar dplyr/tidyverse syntax
- If certain steps in your pipeline are not compatible with DuckDB, data might be collected into memory and the pipeline contonous nevertheless
- After filteringen/aggregation etc you can always choose to `collect()` the data and use your ordinary workflow from there

Fater in memory?

- DuckDB and `{duckplyr}` makes it possible to work with data which does not fit into memory
- What if data would fit?
  - You can use tibbles and dplyr etc of course …
    - … but it might be terribly slooooooooooooow … for big enough data
    - (Although dplyr also gets some speed improvementsfrom time to time)
- `{data.table}` is much more efficient!

data.table

- Share basic properties with data frames (`dt[i, j]; merge()`)

- Has been around since 2006 (stable and well used, even older than tidyverse)
- faster alternatives to base: `fifelse()`, `fcase()`, `forder()`, `fread()`, `fcoalesce()`, `as.IDate()`, `%chin%`
- C and GForce (some difficulties on Mac ...)
- integer based keys and indices (compare to data bases)
- Reference semantics (no need to copy on modification)
- Very flexible and compact (but perhaps unusual) syntax

Example

```
library(data.table)
dt <- NHANES::NHANES
setDT(dt)
setkey(dt, ID) # Use the ID column as index
# How many adults by gender do we have
dt[Age >= 18, .N, Gender]
```

```
   Gender     N
   <fctr> <int>
1:   male  3686
2: female  3795
```

```
# Mean BMI by gender
dt[Age >= 18, mean(BMI, na.rm = TRUE), Gender]
```

```
   Gender       V1
   <fctr>    <num>
1:   male 28.67528
2: female 28.66978
```

```
# Convert all factors to characters
dt[, names(.SD) := lapply(.SD, as.character), .SDcols = is.factor]
```

joins

```
x[i, on, nomatch]
| |  |   |
| |  |   \__ If NULL only returns rows linked in x and i tables
| |  \____ a character vector or list defining match logic
| \_____ primary data.table, list or data.frame
\____ secondary data.table
```

Join type	data.table	SQL	dplyr
Right join	`DT1[DT2]`	RIGHT JOIN	`right_join(DT2, DT1)`
Left join	`DT2[DT1]`	LEFT JOIN	`left_join(DT1, DT2)`
Inner join	`DT1[DT2, nomatch = 0]`	INNER JOIN	`inner_join(DT1, DT2)`
Full join	`merge(DT1, DT2, all = TRUE)`	FULL OUTER JOIN	`full_join(DT1, DT2)`
Anti join	`DT1[!DT2]`	NOT EXISTS	`anti_join(DT1, DT2)`
Semi join	`DT1[DT2, nomatch = 0][, unique(.SD)]`	EXISTS	`semi_join(DT1, DT2)`
Rolling join	`DT1[DT2, on = "date", roll = TRUE]`	—	`join_by()` + `rolling`
Non-equi join	`DT1[DT2, on = .(x >= lower, x <= upper)]`	—	`join_by(x >= lower, x <= upper)`
Update join	`DT1[DT2, value := i.value]`	UPDATE ... JOIN	`DT1 %>% left_join(DT2) %>% mutate(...)`

Bibliography

[1] A. Nguyen, *Hands-on healthcare data: taming the complexity of real-world data*, First edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly, 2022.

[2] H. Wickham, M. Çetinkaya-Rundel, and G. Grolemund, "R for Data Science (2e)." [Online]. Available: https://r4ds.hadley.nz/

[3] S. R. Fenk, K. Furu, and I. J. Bakken, "Improve data management in register-based research: Transition from CSV to Parquet," doi: 10.1101/2025.10.15.25337992.

[4] *Data analysis using data.table*. 2026. [Online]. Available: https://cran.r-project.org/web/packages/data.table/vignettes/datatable-intro.html

[5] V. Arel-Bundock, "data.table vs. base vs. dplyr – Vincent Arel-Bundock." [Online]. Available: https://arelbundock.com/posts/dt_tb_df/index.html